Election Analytics Platform for Portugal

Advanced Topics in Databases Practical Assignment

Rui Coelho, up202106772 Ricardo Ribeiro, up202104806

June 2026

Abstract

This project implements a database-centred analytics platform for Portuguese local elections. The system loads official CNE election spreadsheets for the 2013, 2017, 2021, and 2025 Autarquicas into PostgreSQL/PostGIS through a rerunnable ETL pipeline, normalizes the data into an operational schema, populates a dimensional warehouse, and exposes the results through a thin Flask application using explicit SQL through `psycopg2`. The database includes functions, PL/pgSQL procedures, triggers, materialized views, analytical SQL, D'Hondt mandate allocation, historical comparison queries and views, and CAOP administrative geometries for district and municipality-level map visualization.

1 Scope and Data Sources

The project focuses on Portuguese local elections (*Eleições Autárquicas*). The assignment recommends the 2021 election as a baseline, so the first implementation used Autarquicas 2021. The project was then extended with the official CNE packages for Autarquicas 2013, 2017, and 2025 to support historical comparisons across four local election cycles.

The official election data sources are:

- `al2013_mapaoficial_retif`: official CNE 2013 results package, converted from legacy `.xls` to `.xlsx`;
- `al2017_mapaoficial_retif02_01out2018`: official CNE 2017 results package, converted from legacy `.xls` to `.xlsx`;
- `2021al_mapa_oficial`: official CNE 2021 results package;
- `2025al-mapa-oficial_retificado`: official CNE 2025 results package including the published correction.

Each package contains the same logical maps:

- `mapa_1_resultados.xlsx` or `Parte1_resultados*.xlsx`: registered voters, voters, blank votes, null votes, and votes per candidacy;
- `mapa_anexo.xlsx` or `Parte4_anexo*.xlsx`: local resolution of labels such as `[A]`, `[B]`, and citizen-group candidacies;
- `mapa_2_perc_mandatos.xlsx` or `Parte2_perc_mandatos*.xlsx`: official vote percentages and mandates;
- `mapa_3_eleitos.xlsx` or `Parte3_eleitos*.xlsx`: elected members by territory, organ, and list.

For administrative boundaries, the project uses DGT CAOP GeoPackage files. The available files are CAOP 2025 rather than a separate CAOP file for each election year. This is a limitation, but it is a compatible official DGT administrative-boundary dataset and is explicitly documented. The election facts remain year-specific; the geometries are used for visualization and spatial storage.

2 Architecture

The system is organized around the database. Python is used for ETL and as a thin web layer, but the analytical logic remains explicit in SQL and PL/pgSQL.

- `staging`: raw CNE rows transformed from Excel into relational staging tables;
- `election`: normalized operational schema;
- `dw`: dimensional warehouse for analytical queries;
- `app`: Flask frontend using `psycopg2`;
- `etl`: rerunnable CNE and CAOP loaders.

The loading sequence is: parse CNE Excel files, copy rows into staging, call the parameterized `election.load_from_staging()`

procedure for the selected election, call `dw.refresh_from_operational()`, and refresh materialized views used by the frontend.

The repository structure follows the assignment recommendation. SQL files are placed under `sql/`; Python ETL scripts under `etl/`; the Flask frontend under `app/`; processed data checks under `data/processed/`; and documentation under `docs/`. The raw CNE spreadsheet packages are kept in separate year folders. CAOP GeoPackages are kept in separate CAOP directories and loaded through the spatial ETL script.

3 Operational Model

The operational schema is normalized around elections, organs, territories, candidacies, turnout, votes, seats, and elected members. Parties, coalitions, and citizen groups are unified under `candidacies`, because the CNE spreadsheets mix fixed party columns with local labels such as `[A]` and `[D]`.

Important constraints include unique election codes, unique candidacies per election/organ/territory/source reference, unique vote and seat results per candidacy, and foreign keys from result tables to the core entities. Indexes support territory drill-down, result ranking, spatial lookup through a GiST index, and frontend queries by election, organ, and territory.

The territory hierarchy is represented through `parent_code`, `district_code`, and `municipality_code`. This gives direct access to common drill-down paths without denormalizing result measures. For mainland Portugal, district codes use the pattern `DD0000`; Madeira is represented as `300000`; and the Azores as `400000`. This coding decision reconciles CNE election groupings with CAOP island and regional layers.

The candidacy model is intentionally local to a territory and organ. This is necessary because a label such as `[A]` has different meanings in different municipalities, and even party coalitions can be local. The schema therefore stores both the source reference and the resolved sigla/name.

4 ETL Pipeline

The CNE ETL is implemented in `etl/load_cne_2021.py`. Despite the historical filename, the loader is now parameterized by source folder, election code, election name, and election date. It is rerunnable: staging tables are truncated for each import, the selected election is deleted and reloaded in the operational schema, and the warehouse/materialized views are refreshed from the full operational schema. The project also includes a small XLSX reader in `etl/xlsx_reader.py` to avoid depending on Excel-specific tooling for parsing the CNE files.

The ETL performs several cleaning and reconciliation steps:

- normalizes territorial codes to six characters;
- converts numeric strings into integers or numeric percentages;
- detects candidate columns dynamically, because the 2013, 2017, 2021, and 2025 files do not have identical party blocks;
- resolves local coalition and citizen-group labels using `mapa_anexo.xlsx`;
- stores all candidacies in one structure with a type: party, coalition, or citizen group;
- handles region/district code differences for the mainland, Madeira, and the Azores;
- loads official percentages and mandates for validation against calculated results.

The current parsed counts are shown in Table 1.

Table 1: CNE ETL row counts by election.

Election	Result rows	Vote rows	Elected parsed
AL2013	3738	12152	35579
AL2017	3729	12069	35709
AL2021	3719	12277	35491
AL2025	3897	12839	36521

The ETL uses PostgreSQL `COPY` through `psycopg2` for staging inserts, which is faster and simpler than row-by-row inserts. The transformation from staging to operational tables is then handled inside the database by PL/pgSQL. This separation keeps Python responsible for file parsing and PostgreSQL responsible for relational reconciliation, constraints, and analytical preparation.

The CAOP ETL is implemented separately in `etl/load_caop.py`. It reads GeoPackage layers whose names contain `municipios`, `distritos`, and `freguesias`, reprojects all geometries to EPSG:3763, normalizes territorial codes, dissolves duplicate administrative units where necessary,

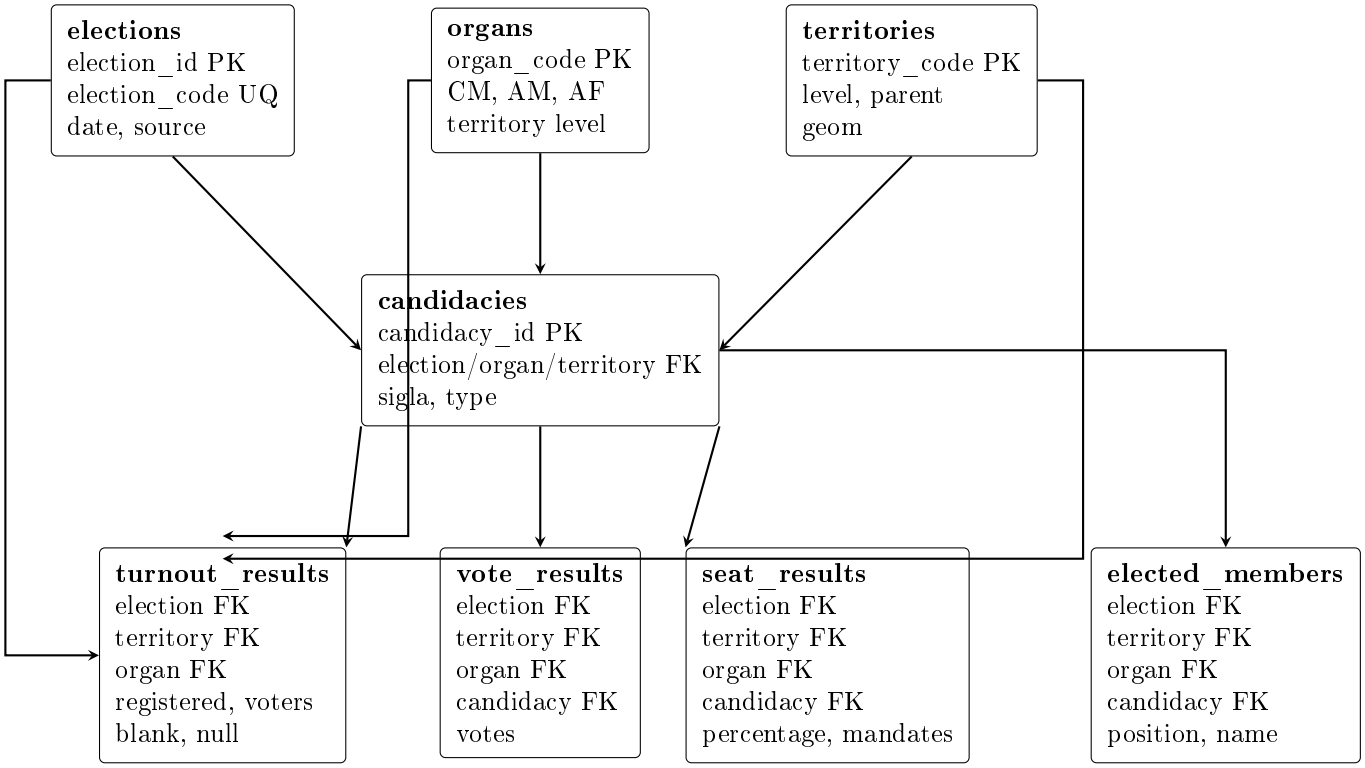

Figure 1: Simplified operational ER diagram. Primary entities are elections, organs, territories, and candidacies; result tables reference these dimensions.

and updates the existing territory rows. This avoids inserting geometries that are not linked to election results.

5 Data Warehouse

The warehouse follows a star-schema style. The central fact table is `dw.fact_election_results`, with dimensions for election, organ, territory, and candidacy. This supports comparisons across territories, candidacies, and election organs while keeping analytical queries simpler than the fully normalized operational schema.

Materialized views are used for frontend performance and repeated analytical access:

- `election.mv_result_summary`;
- `election.mv_national_totals`;
- `election.mv_territory_winners`.

The fact table stores both candidate measures and turnout measures. Although turnout is not candidate-specific, keeping it in the fact table simplifies dashboard queries because each result row can be displayed with the corresponding context. The normalized operational table remains the authoritative source, while the warehouse optimizes analytical access.

The warehouse refresh is destructive and rerunnable: dimensions and facts are truncated and rebuilt from the operational schema. This is acceptable for the project scope because the pipeline loads a fixed official dataset rather than continuously updated production data.

6 SQL Programming

The database contains SQL functions, PL/pgSQL procedures, and triggers. The function `election.vote_share` computes vote share from votes and valid votes. The function `election.turnout_rate` computes turnout. The function `election.dhondt_allocate` implements the D'Hondt method using generated divisors and a ranking window function.

The procedure `election.load_from_staging()` transforms staged CNE rows into the operational schema. The procedure `dw.refresh_from_operational()` rebuilds the warehouse and refreshes materialized views.

Triggers enforce consistency:

- turnout values cannot be negative, and blank plus null votes cannot exceed voters;
- vote results must match the candidacy scope;

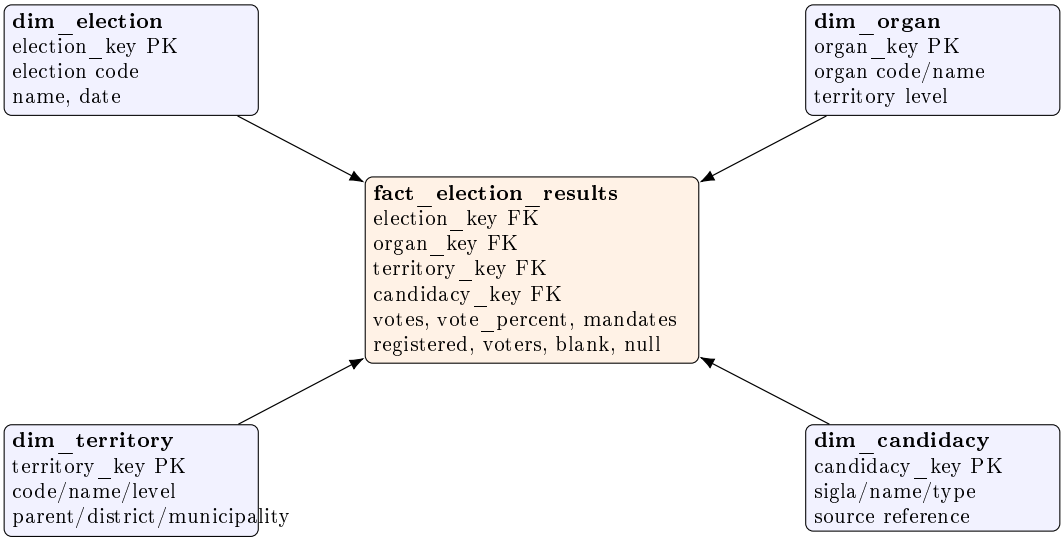

Figure 2: Warehouse star schema. The fact table combines vote, mandate, and turnout measures with dimensions for election, organ, territory, and candidacy.

- seat results must match the candidacy scope and cannot have negative mandates.

The D'Hondt implementation is intentionally implemented in SQL rather than Python. It creates all quotients using `generate_series(1, seats)`, ranks them with a window function, and returns the first `n` quotients. This makes the allocation transparent, testable, and reusable by other SQL queries.

7 Analytical SQL

The file `sql/05_analytical_queries.sql` contains examples required by the assignment:

- D'Hondt allocation;
- three window-function queries;
- one `GROUP BY ROLLUP` query;
- one `GROUP BY CUBE` query;
- advanced aggregation using `FILTER` and `jsonb_agg`.

The extension file `sql/08_historical_comparisons.sql` adds multi-year analytical queries over Autarquicas 2013, 2017, 2021, and 2025. It compares turnout changes by municipality, national vote-share trends by political family, municipalities where the winning list changed, and seat changes across election cycles. The same historical comparison logic is also exposed through the Flask frontend so it can be demonstrated without manually running SQL.

The analytical queries support municipality rankings, national vote shares, turnout rankings inside districts, candidate-type subtotals, and district-level winner summaries.

The `ROLLUP` query produces totals at several levels in one result: candidate totals per district, district totals, and the global total. The `CUBE` query compares votes and mandates across candidate type and district, including subtotals for both dimensions. These queries demonstrate that the warehouse and operational views support aggregate analysis beyond simple result lookup.

8 Spatial Data and Frontend

The `election.territories` table stores geometries as `geometry(MultiPolygon, 3763)`. The CAOP loader reads GeoPackage layers for municipalities, districts/regions, and parishes, reprojects them to EPSG:3763, and updates the territory table.

The loaded geometry coverage is:

Table 2: Loaded CAOP geometry coverage.

Level	Total	With geometry
District/region	20	20
Municipality	308	308
Parish	3394	3241

The assignment minimum is satisfied because all district/region and municipality geometries are

loaded. Parish coverage is partial because CAOP 2025 does not match every historical parish code exactly, especially when comparing elections before and after parish changes.

The frontend is a Flask application. It uses explicit SQL through `psycopg2`; no ORM hides the database logic. The homepage lets the user select the election and organ, then shows a Leaflet municipality map colored by the winning candidacy. The municipality result page shows turnout, a results table, vote-share chart, seat-distribution chart, and elected members when available. The `/compare` page lets the user compare two loaded local-election years and displays political-family vote-share trends, seat changes, turnout changes, and municipalities where the winning list changed.

The map endpoint returns a GeoJSON feature collection generated from PostGIS using `ST_AsGeoJSON`, `ST_Transform`, and `ST_SimplifyPreserveTopology`. Geometries are stored in the Portuguese projected CRS for database work and transformed to EPSG:4326 for Leaflet display. Each feature includes municipality code, name, district/region name, winning sigla, vote share, mandates, and turnout.

The frontend has deliberately limited responsibilities. It does not compute election metrics itself; it only sends SQL queries to the database and renders returned values. Charts are produced with Plotly in Python and embedded into the templates. This follows the assignment requirement that the database component remains central.

9 Validation

The database was loaded and validated on a local PostgreSQL/PostGIS database named `tabd`. The first load command creates the schema and loads the baseline election:

```
.venv/bin/python etl/load_cne_2021.py --setup \
  --dsn "dbname=tabd" \
  --source-dir 2021al_mapa_oficial \
  --election-code AL2021 \
  --election-date 2021-09-26
```

The 2013, 2017, and 2025 packages are then loaded with the same script but without `-setup`. The validation file `sql/07_validation_queries.sql` checks staging, operational, warehouse, materialized view, and D'Hondt results. The historical comparison file `sql/08_historical_comparisons.sql` checks the four loaded elections.

Key validated counts:

Table 3: PostgreSQL validation counts by election.

Election fact	Rows
`AL2013 vote_results`	12152
`AL2017 vote_results`	12069
`AL2021 vote_results`	12277
`AL2025 vote_results`	12839
`dw.fact_election_results`	49337

For Agueda's municipal council, the D'Hondt calculation matched the official mandates: PPD/PSD.MPT received 4 mandates, PS received 2, and CDS-PP received 1.

The Flask map endpoint was also tested through the Flask test client. The homepage returned HTTP 200, the detailed result page for Agueda returned HTTP 200, the historical comparison page returned HTTP 200, and the GeoJSON endpoint returned 308 municipality features. This confirms that the frontend can query election results, historical comparison data, and loaded CAOP geometries.

10 Implementation Files

The most relevant implementation files are:

- `sql/00_extensions_and_schemas.sql`: schema reset and PostGIS extension;
- `sql/01_staging_schema.sql`: CNE staging tables;
- `sql/02_operational_schema.sql`: normalized operational schema;
- `sql/03_warehouse_schema.sql`: warehouse dimensions and fact table;
- `sql/04_functions_views_triggers.sql`: SQL functions, procedures, triggers, views, and D'Hondt;
- `sql/05_analytical_queries.sql`: assignment analytical SQL examples;
- `sql/06_materialized_views.sql`: cached frontend/analytics views;
- `sql/07_validation_queries.sql`: validation queries;
- `sql/08_historical_comparisons.sql`: historical comparison queries for 2013, 2017, 2021, and 2025;

- `etl/load_cne_2021.py`: CNE election ETL;
- `etl/load_caop.py`: CAOP geometry ETL;
- `app/app.py` and `app/db.py`: Flask routes and explicit SQL query helpers, including historical comparison helpers.

11 Limitations and Extensions

The project implements several optional extensions suggested by the assignment: historical comparisons across four election years, support for parish-level result rows, materialized views and precomputed aggregates, and richer frontend interaction including map drill-down and a historical comparison page.

The main limitation is the use of CAOP 2025 geometries as one common boundary layer for all elections. This is acceptable as a documented newer compatible official boundary source, but a perfect historical comparison would load CAOP boundaries matching each election year. Parish geometries are partial because some parish codes changed across the election cycles.

Another limitation concerns elected-member rows. Some elected-member records do not match a normalized candidacy and territory exactly, mostly because of parish-level inconsistencies in list labels/codes. This does not affect votes, turnout, mandates, D'Hondt, warehouse facts, or municipality-level analysis. The official AL2013 package also contains one parish-level turnout anomaly, Covelas in Povoa de Lanhoso for `AF`, where voters exceed registered voters by two; the project preserves the official values and documents the anomaly.

The project does not implement support for multiple election types such as legislative or presidential elections. That extension would require additional organ and territory modelling because local elections use municipal and parish organs, while legislative elections use electoral circles and different seat allocation scopes. Future work could also add party filters, year-specific boundary layers, and election-type-specific frontend views.

12 Contributions

Rui Coelho (up202106772) worked on data inspection, ETL validation, PostgreSQL/PostGIS loading, frontend testing, and documentation.

Ricardo Ribeiro (up202104806) worked on schema design, analytical SQL, D'Hondt validation, report review, and presentation preparation.

AI-assisted programming and documentation support was used during development. The final database design, code, SQL logic, and report contents should be reviewed and understood by both group members before submission.

13 Reproducibility

The project can be reproduced by installing dependencies, loading the CNE data, loading CAOP geometries, and running the Flask app:

```
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/python etl/load_cne_2021.py \
  --setup --dsn "dbname=tabd" \
  --source-dir 2021al_mapa_oficial \
  --election-code AL2021 \
  --election-date 2021-09-26
.venv/bin/python etl/load_cne_2021.py \
  --dsn "dbname=tabd" \
  --source-dir al2013_mapaoficial_retif \
  --election-code AL2013 \
  --election-date 2013-09-29
.venv/bin/python etl/load_cne_2021.py \
  --dsn "dbname=tabd" \
  --source-dir al2017_mapaoficial_retif02_01out2018
  --election-code AL2017 \
  --election-date 2017-10-01
.venv/bin/python etl/load_cne_2021.py \
  --dsn "dbname=tabd" \
  --source-dir 2025al-mapa-oficial_retificado \
  --election-code AL2025 \
  --election-date 2025-10-12
.venv/bin/python etl/load_caop.py \
  --caop-dir . --dsn "dbname=tabd"
.venv/bin/flask --app app/app.py run
```